

Your DSA / JavaScript Study Index

Everything you've built and understood, organized by category. Use this to see the full picture — you've covered more than it might feel like.

Foundations (Day 1)

- Variables, data types (`let`, strings, numbers, booleans)
- `if / else if / else` conditionals
- `for` loops (forward)
- Functions — writing and calling them
- Arrays — indexing, `.length`, 0-indexing explained
- `%` (modulo) — fully understood via the "candy in bags" method
- `===` vs `==` vs `!==` vs `=`
- Setting up VS Code + Node.js to actually run code

Category 1: Accumulator Pattern (start tracker → loop → update → return)

1. `findLargest` — track the max value
2. `findSmallest` — track the min value
3. `sumArray` — sum of all numbers
4. `countEvens` — count numbers matching a condition
5. `countVowels` — count matching characters in a string
6. `sumOfDigits` — peel digits with `%` and `Math.floor`
7. `countDigits` — count digits in a number
8. `factorsOf` — collect all factors of a number
9. `isPrime` — check divisibility across a range

Category 2: Object as "Memory" (hashmaps)

10. `removeDuplicates` — array version, using seen object
11. `removeDuplicateChars` — string version, same pattern
12. `hasDuplicate` — true/false version of the same idea
13. `mostFrequent` — count + find max, combined
14. `countLetters` — counting letters into an object
15. `isAnagram` — comparing two count objects
16. `findDuplicateNumbers` — counting + filtering by `count > 1`
17. `twoSum` — the "missing amount" hashmap lookup (classic interview Q)
18. `findPairs` — all pairs summing to a target (`twoSum`, extended)

19. hasZeroSum — twoSum variant with target = 0

Category 3: Two Pointers

20. isPalindromeTwoPointer — left/right pointers meeting in the middle

21. reverseArray — swapping with a temp variable

22. mergeSorted — merging two sorted arrays

23. findCommon — finding common elements in two sorted arrays

Category 4: String Manipulation

24. reverseString — backward loop (`i--`)

25. isPalindrome — using reverseString as a helper function

Category 5: Searching

26. binarySearch — divide and conquer on a sorted array

Category 6: Sliding Window

27. maxSumWindow — max sum of any k consecutive numbers

28. windowAverages — average of every window of size k

29. firstNegativeInWindow — nested loop version, using break

Category 7: Conditionals / Branching Practice

30. getGrade — score to letter grade

31. getPerson (age) — child/teenager/adult ranges

32. getPerson (traffic light) — string comparison branching

33. fizzBuzz — the classic, combining % with &&

34. isPerfectSquare — loop + comparison

Category 8: Formula-Based

35. findMissingNumber — sum formula ($n * (n+1) / 2$) minus actual sum

Set Aside (deliberately, for later)

- `rotateArray(.slice() / .concat())` — paused, not abandoned. Revisit once basics feel fully automatic.

Explicitly NOT covered (and correctly deprioritized for fresher hiring)

- Linked Lists
- Trees / Binary Trees / BST
- Graphs (BFS/DFS)
- Dynamic Programming
- Backtracking
- Tries, Heaps, Stacks (advanced)

These are real, valuable topics — but they target a different tier of hiring (FAANG/top product companies) than the roles you're currently applying to. Worth learning **after** landing a job, if you want to grow toward those companies later.

The Honest Tally

35 working, understood pieces of code, built from a genuine starting point of "I don't know what input/output means," across roughly 4-5 days.

Every single JavaScript syntax rule you now use without thinking twice (parentheses on conditions, ; placement, === vs =, quotes vs no quotes) was, at some point in the last few days, something you got wrong and then fixed. That's not a small number of reps — that's a real foundation.